Programmation dynamique 2

Prérequis :

- Récursivité
- Principe de la programmation dynamique (activité C57)

Objectifs :

- appliquer la programmation dynamique :
  - à l'alignement de séquences
  - à la suite de Fibonacci

PARTIE II : Alignement de séquences

Introduction

L'alignement de séquences est très utilisée en **bio-informatique**. On cherche à représenter plusieurs séquences de macromolécules biologiques (**ADN, ARN ou protéines**) les unes sous les autres en alignant des motifs similaires.

```
------D-PGDF--DRNVPRICGVCGDRATGFHFNAMTCEGCKGFFRRSMKRKA--LFTCP-FNGDCRITKDNRRHCQACRLKRCVDIGMMKEFILTD
IRPQKRK-KGPAP-KMLGNELCSVCGDKASGFHYNVLSCEGCKGFFRRSVIKGA--HYICH-SGGHCPMDTYMRRKCQECRLRKCRQAGMREECVLSE
SVPGKPS-VNADE-EVGGPQICRVCGDKATGYHFNVMTCEGCKGFFRRAMKRNA--RLRCPFRKGACEITRKTRRQCQACRLRKCLESGMKKEMIMSD
EPERKRK-KGPAP-KMLGHELCRVCGDKASGFHYNVLSCEGCKGFFRRSVVRGGARRYACR-GGGTCQMDAFMRRKCQQCRLRKCKEAGMREQCVLSE
PVTKKPRMGASAG-RIKGDELCVVCGDRASGYHYNALTCEGCKGFFRRSITKNA--VYKCK-NGGNCVMDMYMRRKCQECRLRKCKEMGMLAECMYTG
QTEEKKC-KGYIPSYLDKDELCVVCGDKATGYHYRCITCEGCKGFFRRTIQKNLHPSYSCK-YEGKCVIDKVTRNQCQECRFKKCIYVGMATDLVLDD
----SPS-PPPPP---RVYKPCFVCNDKSSGYHYGVSSCEGCKGFFRRSIQKNM--VYTCH-RDKNCIINKVTRNRCQYCRLQKCFEVGMSKEAVRND
----PPS-PLPPP---RVYKPCFVCQDKSSGYHYGVSACEGCKGFFRRSIQKNM--IYTCH-RDKNCVINKVTRNRCQYCRLQKCFEVGMSKESVRND
----PPS-PPPLP---RIYKPCFVCQDKSSGYHYGVSACEGCKGFFRRSIQKNM--VYTCH-RDKNCIINKVTRNRCQYCRLQKCFEVGMSKESVRND
```

Alignement de séquences de récepteurs nucléaires. Les acides aminés conservés sont surlignés en bleu et en vert. Par endroits, on a inséré des trous, symbolisés par des tirets « - », pour permettre un alignement optimal
Source : Par Fdardel — Travail personnel, CC BY-SA 3.0, https://commons.wikimedia.org/w/index.phpcurid=17642016

On peut par exemple :

- Prédire les fonctions d'une protéine par homologie avec une protéine déjà connue. En effet, deux protéines dont les séquences présentent des ressemblances ont probablement des fonctions apparentées
- Interpréter les alignements dans le cadre de la **théorie de l'évolution** comme dérivant d'un **ancêtre commun** et les divergences comme des mutations. En effet, trouver des fragments de séquences

identiques pour deux génomes donnés peut **prouver une filiation entre les deux êtres vivants** auxquels ils correspondent.

Pour en savoir plus : http://lhommeenquestions.museedelhomme.fr/si-je-partage-98-mes-genes-chimpanze-pourquoi-je-n-partage-que-50-mon-frere

Énoncé du problème

Le génome humain est donc codé avec quatre bases nucléiques ou nucléotides : Adénine, Cytosine, Thymine, Guanine. On peut le représenter par des très longs mots sur l'alphabet {A, C, T, G}. La comparaison de deux génomes repose sur la comparaison d'une suite de nucléotides.

On peut formaliser l'alignement de séquences d'ADN en ces termes : étant données deux chaînes de caractères, il s'agit de mettre en correspondance « le plus possible » les caractères de la 1re chaîne avec ceux de la seconde, en conservant l'ordre des caractères mais en s'autorisant à insérer des trous dans l'une ou l'autre chaîne.

Exemple : on peut aligner les chaînes 'AGTTC' et 'AGCTC' ainsi, où le caractère – est utilisé pour désigner un trou :

Séquence 1	A	G	T	-	T	C
Séquence 2	A	G	-	C	T	C

Bien sûr, il existe d'autres façons d'aligner ces deux séquences. En voici une autre :

Séquence 1	A	G	-	T	T	C
Séquence 2	A	G	C	T	-	C

La seule contrainte est que l'on doit utiliser tous les caractères de chaque chaîne, dans l'ordre, et que l'on n'aligne jamais deux trous (c'est-à-dire deux caractères -). Parmi tous les alignements possibles, on cherche celui qui donne le **score maximal**.

Le score est ainsi calculé :

- l'alignement de deux caractères identiques donne un point
- l'alignement de deux caractères différents, ce qui inclut l'alignement avec le caractère -, enlève un point

Ainsi, les deux alignements proposés dans l'exemple précédent obtiennent chacun le score 2, car on a 4 caractères correctement alignés et deux mauvais alignements. Ils donnent tous les deux le score maximal.

Exercice 1 :

Soit les deux séquences seq1 = 'ACTACGAACTAG' et seq2 = 'GACTAGACTAAGC'.
Donne un alignement possible et calcule son score. Compare avec tes camarades pour savoir si tu as trouvé celui de score maximal.
On note que les deux séquences n'ont pas forcément la même longueur.

Recherche d'une formule de récurrence

Comme pour le problème du rendu de monnaie de la partie I, nous allons commencer par chercher une solution récursive à notre problème (mais cette fois sans écrire le code Python), puis en améliorer l'efficacité grâce à la programmation dynamique.

Reprenons l'exemple des mots 'AGTTC' et 'AGCTC' et intéressons-nous à leur dernier caractère. Ici, il s'agit du C dans les deux cas. Il y a alors trois possibilités :

- soit ces deux C sont alignés, on marque un point et on doit aligner les mots 'AGTT' et 'AGCT' ;
- soit le C final de 'AGTTC' est aligné avec -, on perd un point et on doit aligner le mot 'AGTT' avec le mot 'AGCTC' ;
- soit le C final de 'AGCTC' est aligné avec -, on perd un point et on doit aligner le mot 'AGTTC' avec le mot 'AGCT'.

Comme il n'y a pas d'autres cas possibles, le score maximal sera le maximum des trois scores obtenus. Et comme on s'est ramené à des mots plus petits, plus précisément à une longueur totale des deux mots strictement plus petite, on peut procéder récursivement.

Ainsi, dans le 1er cas on calcule le score maximal de 'AGTT' et 'AGCT', dans le 2e cas de 'AGTT' et 'AGCTC', et dans le 3e cas de 'AGTTC' et 'AGCT'. A chaque fois, il y a à nouveau trois cas de figure, qui

nous ramènent à des mots plus courts. Et ainsi de suite. Le processus termine lorsque l'un des deux mots n'a plus de caractère.

On pourrait ainsi écrire une fonction récursive `alignement` qui applique l'algorithme ci-dessus. Mais exactement comme pour le rendu de monnaie, elle serait très inefficace, car elle passerait son temps à recalculer la même chose. En effet, dans le 2e cas, le calcul de l'alignement de AGTT' et 'AGCTC' va nous ramener en particulier au calcul de l'alignement de AGTT' et 'AGCT', déjà fait dans le 1er cas, et dans le 3e cas, le calcul de l'alignement de 'AGTTC' et 'AGCT' nous rmène une 3e fois au calcul de l'alignement de AGTT' et 'AGCT' !!

Ce phénomène de duplication des calculs va se répéter à chaque étape, provoquant une **explosion exponentielle des appels récursifs et donc du temps de calcul.** Nous allons donc améliorer notre programme en évitant les calculs redondants grâce à la programmation dynamique.

Programmation dynamique

Nous allons utiliser un tableau à deux dimensions dans lequel on va stocker, pour chaque paire de mots, le score maximal. Plus précisément, pour deux entiers *i* et *j*, on va stocker dans la case (*i*, *j*) de ce tableau le score maximal de l'alignement des i premiers caractères du premier mot et des j premiers caractères du second mot.

Exemple : on reprend les chaînes seq1 = 'AGTTC' et seq2 = 'AGCTC'.

Le tableau obtenu au final (avec les bordures), noté sc, est celui-ci :

			A	**G**	**C**	**T**	**C**
	i / j	0	1	2	3	4	5
	0	0	– 1	– 2	– 3	– 4	– 5
A	1	– 1	1	0	– 1	– 2	– 3
G	2	– 2	0	2	1	0	– 1
T	3	– 3	– 1	1	1	2	1
T	4	– 4	– 2	0	0	2	1
C	5	– 5	– 3	– 1	1	1	3

Étapes :

- on commence par initialiser la 1re ligne (sc[0][j]) et la 1re colonne (sc[i][0]). Dans les deux cas, il s'agit de l'alignement avec un mot vide (de longueur 0), pour lequel il faut supprimer tous les caractères, ce qui produit un score égal à l'opposé de la longueur du mot proposé ;
- on complète en progressant dans le sens des indices croissants et en utilisant la formule de récurrence suivante : **sc[i][j] = max(– 1 + sc[i – 1][j], – 1 + sc[i][j – 1], ε + sc[i – 1][j – 1])**

  avec ε = 1 si seq1[i – 1] et seq2[j – 1] coïncident, sinon – 1, et pour i > 0, j > 0.

  case rouge : *i = 2 et j = 3* : seq1[i – 1] = seq1[1] = 'G' ne coïncide pas avec seq2[j – 1] = seq2[2] = 'C' donc sc[2][3] = max(– 1 + sc[1][3], – 1 + sc[2][2], – 1 + sc[1][2]) soit

  sc[2][3] = max(– 1 + (– 1), – 1 + 2, – 1 + 0) = max(– 2, 1, – 1) = 1

  case verte : *i = 4 et j = 4* : seq1[i – 1] = seq1[3] = 'T' coïncide avec seq2[j – 1] = seq2[3] = 'T' donc sc[4][4] = max(– 1 + sc[3][4], – 1 + sc[4][3], + 1 + sc[3][3]) soit

  sc[4][4] = max(– 1 + 2, – 1 + 0, + 1 + 1) = max(1, – 1, 2) = 2

Exercice 2 :

Calculer « à la main » le tableau sc des scores d'alignement pour les deux séquences seq1 = 'CAT' et seq2 = 'CGAT'. Quel est donc le score maximal ?

Exercice 3 :

Soit l'algorithme en langage naturel de l'alignement des séquences utilisant la programmation dynamique :

Entrées : Deux séquences seq1 et seq2
Sorties Le score maximal entre les deux séquences

fonction alignement(seq1, seq2) :

```
n1 ← taille de seq1
n2 ← taille de seq2
sc ← [[0]*(n1+1) pour k allant de 0 à n2]
# Cas de base : 1re ligne et 1re colonne
Pour i=0 à m:  sc[i][0]= – i
Pour j=0 à n:  sc[0][j]= – j
# Le reste
Pour i=1 à m:
   Pour j=1 à n:
      Si seq1[i – 1]==seq2[j – 1] :
         sc[i][j] ← max(sc[i][j−1] – 1, sc[i−1][j] – 1,sc[i−1][j−1] + 1)
      Sinon :
         sc[i][j] ← max(sc[i][j−1] – 1, sc[i−1][j] – 1,sc[i−1][j−1] – 1)
Renvoyer sc[m][n]
```

1. Implémente cet algorithme en Python et enregistre-le sous `C58_ex03.py`.
2. Teste le programme avec seq1 = 'AGTTC' et seq2 = 'AGCTC'.

Pour les plus rapides :

3. Modifie la fonction `rendu_monnaie_rec()` pour qu'elle renvoie le tuple (score maximal, couple solution des séquences de même longueur contenant d'éventuels caractères -).

Aide : il faut ajouter un second tableau `solution` initialisé avec des couples de chaînes vides.

PARTIE III : Suite de Fibonacci

Exercice 4 :

La suite d'entiers 0, 1, 1, 2, 3, 5, 8, 13, 21… où chaque terme s'obtient en additionnant les deux précédents porte le nom de suite de Fibonacci, en hommage à Leonardo Fibonacci mathématicien italien du XIIIe siècle qui l'avait utilisée pour modéliser l'évolution d'une population de lapins.
En langage moderne mathématique, cette suite que l'on notera (F_n) est définie par ses deux premiers termes $F_0 = 0$, $F_1 = 1$ et la relation de récurrence : **pour tout** $\forall n \in \mathbb{N},\ F_{n+2} = F_{n+1} + F_n$

1. Écrire l'algorithme en Python à l'aide d'une fonction récursive simple `fibonacci_rec(n)` permettant le calcul d'un terme de la suite de Fibonacci (*n* étant le rang du terme de la suite de Fibonacci à calculer)

2. Écrire l'algorithme en Python à l'aide d'une fonction itérative (forme Bottom Up) en programmation dynamique `fibonacci_BU(n)` permettant le calcul d'un terme de la suite de Fibonacci (*n* étant le rang du terme de la suite de Fibonacci à calculer).

3. Évaluer le temps mis par les deux algorithmes pour calculer F_{40}. Commenter.

4. Quel est le principal avantage de la programmation dynamique ?

5. Quel inconvénient peut survenir avec l'utilisation de la programmation dynamique ?

6. Évaluer le coût de stockage mémoire de votre algorithme de programmation dynamique de calcul de la suite de Fibonacci pour le terme F_{50}.

On pourra considérer que Python utilise une allocation mémoire moyenne pour ces différents types comme suit (données très approximatives car Python adapte la taille d'allocation mémoire nécessaire à chaque donnée) :

Type	Bornes	Allocation mémoire
Entier court	De – 2 147 483 648 à + 2 147 483 647	4 octets (32 bits)
Entier long	De – 1,84446744…x10^{19} à +1,84446744…x10^{19}	8 octets (64 bits)
Flottant (réel)	De 10^{-308} et 10^{308} (en positif ou négatif)	8 octets (64 bits)

NB : une liste contenant des entiers courts nécessite en général le double d'allocation mémoire de la somme des éléments qui la constitue (adressage des données dans la liste).